

Traq: Estimating the Quantum Cost of Classical Programs

Anurudh Peduri, **Jam Kabeer Ali Khan**, Gilles Barthe, Michael Walter

MAX PLANCK INSTITUTE
FOR SECURITY AND PRIVACY



RUHR
UNIVERSITÄT
BOCHUM

RUB

Motivation

Many quantum algorithms offer speedups over classical ones *, e.g., Grover's algorithm.

How to leverage these speedups in existing classical programs? *Quantization*



How to perform quantization? *Compilation*

*Quantum Algorithms Zoo <https://quantumalgorithmzoo.org/>

Motivation

Many quantum algorithms offer speedups over classical ones *, e.g., Grover's algorithm.

How to leverage these speedups in existing classical programs? *Quantization*



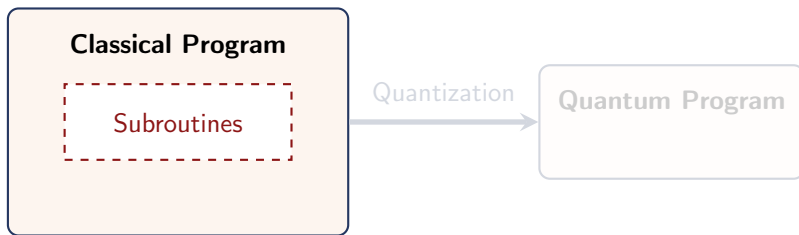
How to perform quantization? *Compilation*

*Quantum Algorithms Zoo <https://quantumalgorithmzoo.org/>

Motivation

Many quantum algorithms offer speedups over classical ones *, e.g., Grover's algorithm.

How to leverage these speedups in existing classical programs? *Quantization*



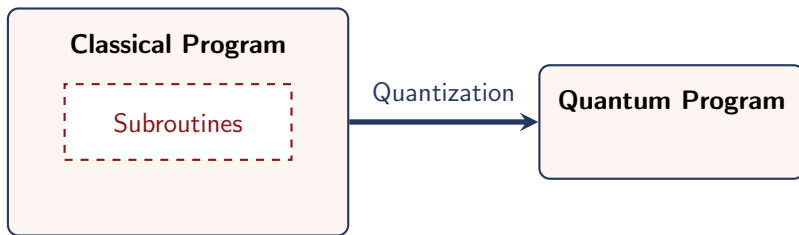
How to perform quantization? *Compilation*

*Quantum Algorithms Zoo <https://quantumalgorithmzoo.org/>

Motivation

Many quantum algorithms offer speedups over classical ones *, e.g., Grover's algorithm.

How to leverage these speedups in existing classical programs? *Quantization*



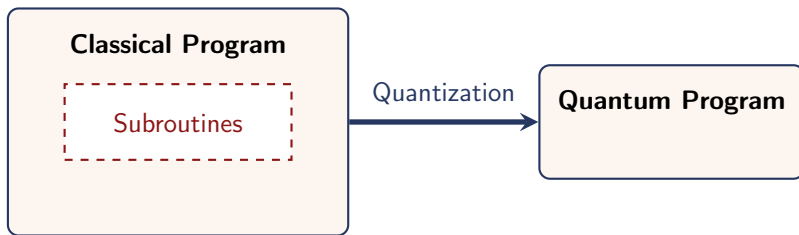
How to perform quantization? *Compilation*

*Quantum Algorithms Zoo <https://quantumalgorithmzoo.org/>

Motivation

Many quantum algorithms offer speedups over classical ones *, e.g., Grover's algorithm.

How to leverage these speedups in existing classical programs? *Quantization*



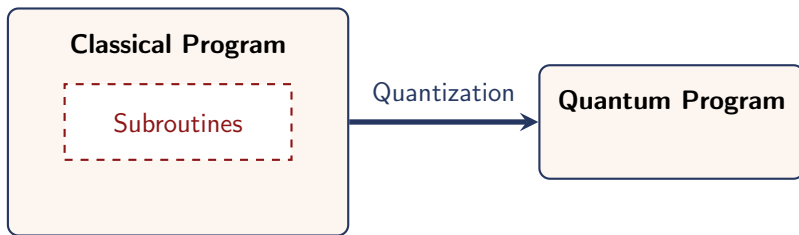
How to perform quantization? *Compilation*

*Quantum Algorithms Zoo <https://quantumalgorithmzoo.org/>

Motivation

Many quantum algorithms offer speedups over classical ones *, e.g., Grover's algorithm.

How to leverage these speedups in existing classical programs? *Quantization*



How to perform quantization? *Compilation*

*Quantum Algorithms Zoo <https://quantumalgorithmzoo.org/>

Challenge: How to estimate the advantage?

How to estimate the advantage of the resulting quantum program?

- ! Execute? Quantum computers not widely available
- ! Simulate? Does not scale!
- ! Manual Analysis? Tedious! Error prone!

Challenge: How to estimate the advantage?

How to estimate the advantage of the resulting quantum program?

- ! Execute? Quantum computers not widely available
- ! Simulate? Does not scale!
- ! Manual Analysis? Tedious! Error prone!

Challenge: How to estimate the advantage?

How to estimate the advantage of the resulting quantum program?

- ! Execute? Quantum computers not widely available
- ! Simulate? Does not scale!
- ! Manual Analysis? Tedious! Error prone!

Challenge: How to estimate the advantage?

How to estimate the advantage of the resulting quantum program?

- ! Execute? Quantum computers not widely available
- ! Simulate? Does not scale!
- ! Manual Analysis? Tedious! Error prone!

Challenge: How to estimate the advantage?

How to estimate the advantage of the resulting quantum program?

- ! Execute? Quantum computers not widely available
- ! Simulate? Does not scale!
- ! Manual Analysis? Tedious! Error prone!

Challenge: How to estimate the advantage?

How to estimate the advantage of the resulting quantum program?

- ! Execute? Quantum computers not widely available
- ! Simulate? Does not scale!
- ! Manual Analysis? Tedious! Error prone!

Challenge: How to estimate the advantage?

How to estimate the advantage of the resulting quantum program?

- ! Execute? Quantum computers not widely available
- ! Simulate? Does not scale!
- ! Manual Analysis? Tedious! Error prone!

Goal

Research Goal

A formal and practical framework to automatically estimate the quantum cost of classical programs.

Automated

No Manual Cost Analysis

Formal

Provable Guarantees

Accessible

No QC expertise required

Goal

Research Goal

A formal and practical framework to automatically estimate the quantum cost of classical programs.

Automated

No Manual Cost Analysis

Formal

Provable Guarantees

Accessible

No QC expertise required

Goal

Research Goal

A formal and practical framework to automatically estimate the quantum cost of classical programs.

Automated

No Manual Cost Analysis

Formal

Provable Guarantees

Accessible

No QC expertise required

Goal

Research Goal

A formal and practical framework to automatically estimate the quantum cost of classical programs.

Automated

No Manual Cost Analysis

Formal

Provable Guarantees

Accessible

No QC expertise required

Traq: A Framework for Quantum Cost Analysis of Classical Programs.

- A Classical (Probabilistic) Source Language, CPL.
- Quantization via Compilation of CPL to a Quantum Programming Language, QPL.
- Source-level Quantum Cost Analysis for CPL programs.
- Provable Guarantees: Compilation and Cost Correctness.

Traq: A Framework for Quantum Cost Analysis of Classical Programs.

- A Classical (Probabilistic) Source Language, CPL.
- Quantization via Compilation of CPL to a Quantum Programming Language, QPL.
- Source-level Quantum Cost Analysis for CPL programs.
- Provable Guarantees: Compilation and Cost Correctness.

Traq: A Framework for Quantum Cost Analysis of Classical Programs.

- A Classical (Probabilistic) Source Language, CPL.
- Quantization via Compilation of CPL to a Quantum Programming Language, QPL.
- Source-level Quantum Cost Analysis for CPL programs.
- Provable Guarantees: Compilation and Cost Correctness.

Traq: A Framework for Quantum Cost Analysis of Classical Programs.

- A Classical (Probabilistic) Source Language, CPL.
- Quantization via Compilation of CPL to a Quantum Programming Language, QPL.
- Source-level Quantum Cost Analysis for CPL programs.
- Provable Guarantees: Compilation and Cost Correctness.

Traq: A Framework for Quantum Cost Analysis of Classical Programs.

- A Classical (Probabilistic) Source Language, CPL.
- Quantization via Compilation of CPL to a Quantum Programming Language, QPL.
- Source-level Quantum Cost Analysis for CPL programs.
- Provable Guarantees: Compilation and Cost Correctness.

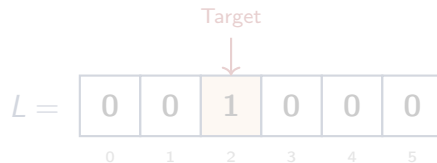
Example 1: Search over List

Problem 1

Given a list L of size N , consisting of 0s and 1s, find if it contains a 1.

$L : \text{Fin}\langle N \rangle \rightarrow \text{Bool}$

```
fn ContainsOne() → Bool do
  b, i ← search[L]; // Primitive
  return b
end
```



- ✓ `search` compiles to **Grover's Algorithm**.
- ✓ Quadratic speedup over classical search.

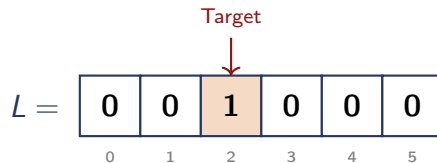
Example 1: Search over List

Problem 1

Given a list L of size N , consisting of 0s and 1s, find if it contains a 1.

$L : \text{Fin}\langle N \rangle \rightarrow \text{Bool}$

```
fn ContainsOne() → Bool do
   $b, i \leftarrow \text{search}[L]; // \text{Primitive}$ 
  return  $b$ 
end
```



- ✓ `search` compiles to Grover's Algorithm.
- ✓ Quadratic speedup over classical search.

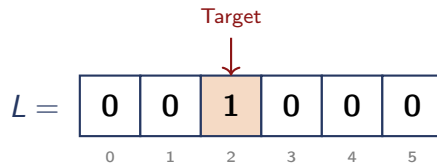
Example 1: Search over List

Problem 1

Given a list L of size N , consisting of 0s and 1s, find if it contains a 1.

$L : \text{Fin}\langle N \rangle \rightarrow \text{Bool}$

```
fn ContainsOne() → Bool do
   $b, i \leftarrow \text{search}[L]; // \text{Primitive}$ 
  return  $b$ 
end
```



- ✓ `search` compiles to **Grover's Algorithm**.
- ✓ Quadratic speedup over classical search.

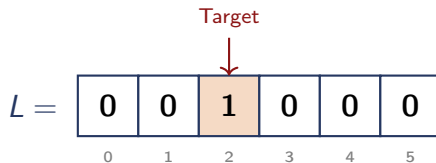
Example 1: Search over List

Problem 1

Given a list L of size N , consisting of 0s and 1s, find if it contains a 1.

$L : \text{Fin}\langle N \rangle \rightarrow \text{Bool}$

```
fn ContainsOne() → Bool do
   $b, i \leftarrow \text{search}[L]; // \text{Primitive}$ 
  return  $b$ 
end
```



- ✓ `search` compiles to **Grover's Algorithm**.
- ✓ Quadratic speedup over classical search.

Example 2: Search over Matrix

Problem 2

Given a $N \times M$ matrix A of 0s and 1s, does it have a row containing all 1s?

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes( $i : \text{Fin}\langle N \rangle$ )  $\rightarrow \text{Bool}$  do  
   $b \leftarrow \text{all}[A(i, \_)]$ ;  //Partial application :  $A(i, \_)$   
  return  $b$ 
```

```
fn HasAllOnesRow()  $\rightarrow \text{Bool}$  do  
   $b, i \leftarrow \text{search}[ \text{IsRowAllOnes}(\_) ]$ ;  
  return  $b$ 
```

$A =$

	0	1	2	3	4
0	0	1	0	0	1
1	1	1	1	1	1

Target Row (All 1s)

- ✓ **search** compiles to Grover's;
IsRowAllOnes acts as the **Oracle**.
- ! Oracles must be reversible: **Unitary**
Compilation ($UU^\dagger = U^\dagger U = I$) is required.

Example 2: Search over Matrix

Problem 2

Given a $N \times M$ matrix A of 0s and 1s, does it have a row containing all 1s?

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes( $i : \text{Fin}\langle N \rangle$ )  $\rightarrow \text{Bool}$  do
   $b \leftarrow \text{all}[A(i, \_)]$ ;  // Partial application :  $A(i, \_)$ 
  return  $b$ 
```

```
fn HasAllOnesRow()  $\rightarrow \text{Bool}$  do
   $b, i \leftarrow \text{search}[ \text{IsRowAllOnes}(\_) ]$ ;
  return  $b$ 
```

$A =$

	0	1	2	3	4
0	0	1	0	0	1
1	1	1	1	1	1

Target Row (All 1s)

- ✓ `search` compiles to Grover's; `IsRowAllOnes` acts as the **Oracle**.
- ! Oracles must be reversible: **Unitary Compilation** ($UU^\dagger = U^\dagger U = I$) is required.

Example 2: Search over Matrix

Problem 2

Given a $N \times M$ matrix A of 0s and 1s, does it have a row containing all 1s?

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes( $i : \text{Fin}\langle N \rangle$ )  $\rightarrow \text{Bool}$  do  
   $b \leftarrow \text{all}[A(i, \_)]$ ;  // Partial application :  $A(i, \_)$   
  return  $b$ 
```

```
fn HasAllOnesRow()  $\rightarrow \text{Bool}$  do  
   $b, i \leftarrow \text{search}[ \text{IsRowAllOnes}(\_) ]$ ;  
  return  $b$ 
```

$A =$

	0	1	2	3	4
0	0	1	0	0	1
1	1	1	1	1	1

Target Row (All 1s)

✓ **search** compiles to Grover's;
IsRowAllOnes acts as the **Oracle**.

! Oracles must be reversible: **Unitary**
Compilation ($UU^\dagger = U^\dagger U = I$) is required.

Example 2: Search over Matrix

Problem 2

Given a $N \times M$ matrix A of 0s and 1s, does it have a row containing all 1s?

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes( $i : \text{Fin}\langle N \rangle$ )  $\rightarrow \text{Bool}$  do  
   $b \leftarrow \text{all}[A(i, \_)]$ ;  //Partial application :  $A(i, \_)$   
  return  $b$ 
```

```
fn HasAllOnesRow()  $\rightarrow \text{Bool}$  do  
   $b, i \leftarrow \text{search}[ \text{IsRowAllOnes}(\_) ]$ ;  
  return  $b$ 
```

$A =$

	0	1	2	3	4
0	0	1	0	0	1
1	1	1	1	1	1

Target Row (All 1s)

- ✓ **search** compiles to Grover's;
IsRowAllOnes acts as the **Oracle**.
- ! Oracles must be reversible: **Unitary**
Compilation ($UU^\dagger = U^\dagger U = I$) is required.

Example 2: Search over Matrix

Problem 2

Given a $N \times M$ matrix A of 0s and 1s, does it have a row containing all 1s?

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes( $i : \text{Fin}\langle N \rangle$ )  $\rightarrow \text{Bool}$  do  
   $b \leftarrow \text{all}[A(i, \_)]$ ;  // Partial application :  $A(i, \_)$   
  return  $b$ 
```

```
fn HasAllOnesRow()  $\rightarrow \text{Bool}$  do  
   $b, i \leftarrow \text{search}[ \text{IsRowAllOnes}(\_) ]$ ;  
  return  $b$ 
```

$A =$

	0	1	2	3	4
0	0	1	0	0	1
1	1	1	1	1	1

Target Row (All 1s)

- ✓ **search** compiles to Grover's;
IsRowAllOnes acts as the **Oracle**.
- ! Oracles must be reversible: **Unitary**
Compilation ($UU^\dagger = U^\dagger U = I$) is required.

Error Analysis: Failure probabilities in Quantum Programs

! Primitives, e.g., [search](#), compile to quantum algorithms that can fail with some probability.

Goal: Bound the total error compositionally.

Error Analysis: Failure probabilities in Quantum Programs

! Primitives, e.g., `search`, compile to quantum algorithms that can fail with some probability.

Goal: Bound the total error compositionally.

```
fn IsRowAllOnes(i) ...  
  b ← all[..];  
  ...
```

```
fn HasAllOnesRow() ...  
  b, i ← search[IsRowAllOnes];  
  ...
```

CPL Source Program

Error Analysis: Failure probabilities in Quantum Programs

! Primitives, e.g., `search`, compile to quantum algorithms that can fail with some probability.

Goal: Bound the total error compositionally.

```
fn IsRowAllOnes(i) ...  
  b ← all[...];  
  ...
```

Error Annotation

→

```
fn HasAllOnesRow() ...  
  b, i ← search[IsRowAllOnes];  
  ...
```

```
fn IsRowAllOnes(i) ...  
  b ← all $\varepsilon_1$ [...];  
  ...
```

```
fn HasAllOnesRow() ...  
  b, i ← search $\varepsilon_2$ [IsRowAllOnes];  
  ...
```

CPL Source Program

Error Analysis: Failure probabilities in Quantum Programs

! Primitives, e.g., `search`, compile to quantum algorithms that can fail with some probability.

Goal: Bound the total error compositionally.

```
fn IsRowAllOnes(i) ...
  b ← all[...];
  ...
```

Error Annotation

→

```
fn HasAllOnesRow() ...
  b, i ← search[IsRowAllOnes];
  ...
```

CPL Source Program

```
fn IsRowAllOnes(i) ...
  b ← allε1[...];
  ...
```

```
fn HasAllOnesRow() ...
  b, i ← searchε2;
  ...
```

↓ Error Analysis

$$\begin{aligned}\varepsilon_{\text{total}} &= \varepsilon_2 + W(N, \varepsilon_2) \sqrt{2\varepsilon_1} \\ &\leq \varepsilon_{\text{max}}\end{aligned}$$

Error Analysis: Failure probabilities in Quantum Programs

! Primitives, e.g., `search`, compile to quantum algorithms that can fail with some probability.

Goal: Bound the total error compositionally.

```
fn IsRowAllOnes(i) ...
  b ← all[...];
  ...
```

Error Annotation

```
fn IsRowAllOnes(i) ...
  b ← allε1[...];
  ...
```

Compilation

```
uproc IsRowAllOnesU(q̄) ...
  call QPε1[...]
  ...
```

```
fn HasAllOnesRow() ...
  b, i ← search[IsRowAllOnes];
  ...
```

→

```
fn HasAllOnesRow() ...
  b, i ← searchε2[...];
  ...
```

→

```
proc HasAllOnesRowQ() ...
  call QPε2[...]
  ...
```

CPL Source Program

↓ Error Analysis

3. Quantum Program

$$\begin{aligned}\varepsilon_{\text{total}} &= \varepsilon_2 + W(N, \varepsilon_2) \sqrt{2\varepsilon_1} \\ &\leq \varepsilon_{\text{max}}\end{aligned}$$

Compiler Correctness: Formal Guarantees

Theorem 1 (General Compiler Correctness)

For every well-formed CPL statement s , the total variation distance between the semantics of the s and the compiled quantum program $Q[s]$ is bounded by error $\text{err}^Q[s]$:

$$\Delta(\llbracket Q[s] \rrbracket_{\text{QPL}}, \llbracket s \rrbracket_{\text{CPL}}) \leq \text{err}^Q[s]$$

+ Unitary compiler correctness

Compiler Correctness: Formal Guarantees

Theorem 1 (General Compiler Correctness)

For every well-formed CPL statement s , the total variation distance between the semantics of the s and the compiled quantum program $Q[s]$ is bounded by error $\text{err}^Q[s]$:

$$\Delta(\llbracket Q[s] \rrbracket_{\text{QPL}}, \llbracket s \rrbracket_{\text{CPL}}) \leq \text{err}^Q[s]$$

+ Unitary compiler correctness

Query Model & Quantum Cost Analysis

Query Model: Counting the number of queries made to a function $f : X \rightarrow Y$.

- Well-established proxy for time complexity.
- Widely used in literature.

Classical Query: A call to function f .

Quantum Query: Invoke Unitary U_f (or its inverse $U_f^\dagger$), where $U_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$

Source-level Quantum Cost Analysis on the CPL programs.

- Havoc cost.
- Input-dependent cost.
- Unitary cost.

Query Model & Quantum Cost Analysis

Query Model: Counting the number of queries made to a function $f : X \rightarrow Y$.

- Well-established proxy for time complexity.
- Widely used in literature.

Classical Query: A call to function f .

Quantum Query: Invoke Unitary U_f (or its inverse $U_f^\dagger$), where $U_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$

Source-level Quantum Cost Analysis on the CPL programs.

- Havoc cost.
- Input-dependent cost.
- Unitary cost.

Query Model & Quantum Cost Analysis

Query Model: Counting the number of queries made to a function $f : X \rightarrow Y$.

- Well-established proxy for time complexity.
- Widely used in literature.

Classical Query: A call to function f .

Quantum Query: Invoke Unitary U_f (or its inverse $U_f^\dagger$), where $U_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$

Source-level Quantum Cost Analysis on the CPL programs.

- Havoc cost.
- Input-dependent cost.
- Unitary cost.

Query Model & Quantum Cost Analysis

Query Model: Counting the number of queries made to a function $f : X \rightarrow Y$.

- Well-established proxy for time complexity.
- Widely used in literature.

Classical Query: A call to function f .

Quantum Query: Invoke Unitary U_f (or its inverse $U_f^\dagger$), where $U_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$

Source-level Quantum Cost Analysis on the CPL programs.

- Havoc cost.
- Input-dependent cost.
- Unitary cost.

Query Model & Quantum Cost Analysis

Query Model: Counting the number of queries made to a function $f : X \rightarrow Y$.

- Well-established proxy for time complexity.
- Widely used in literature.

Classical Query: A call to function f .

Quantum Query: Invoke Unitary U_f (or its inverse $U_f^\dagger$), where $U_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$

Source-level Quantum Cost Analysis on the CPL programs.

- Havoc cost.
- Input-dependent cost.
- Unitary cost.

Query Model & Quantum Cost Analysis

Query Model: Counting the number of queries made to a function $f : X \rightarrow Y$.

- Well-established proxy for time complexity.
- Widely used in literature.

Classical Query: A call to function f .

Quantum Query: Invoke Unitary U_f (or its inverse $U_f^\dagger$), where $U_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$

Source-level Quantum Cost Analysis on the CPL programs.

- Havoc cost.
- Input-dependent cost.
- Unitary cost.

Query Model & Quantum Cost Analysis

Query Model: Counting the number of queries made to a function $f : X \rightarrow Y$.

- Well-established proxy for time complexity.
- Widely used in literature.

Classical Query: A call to function f .

Quantum Query: Invoke Unitary U_f (or its inverse $U_f^\dagger$), where $U_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$

Source-level Quantum Cost Analysis on the CPL programs.

- Havoc cost.
- Input-dependent cost.
- Unitary cost.

Havoc Cost: Upper-bound on program execution with errors.

Goal: Capture worst-case cost in the presence of errors.

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes( $i : \text{Fin}\langle N \rangle$ )  $\rightarrow \text{Bool}$  do  
   $b \leftarrow \text{all}[A(i, \_)]$ ;  
  return  $b$ 
```

```
fn HasAllOnesRow()  $\rightarrow \text{Bool}$  do  
   $b, i \leftarrow \text{search}[\text{IsRowAllOnes}(\_)]$ ;  
  return  $b$ 
```

$$\begin{aligned} & \widehat{\text{HAVOC}}^Q[\text{HasAllOnesRow}] \\ &= W(N, \varepsilon_2) \cdot 2 \cdot \widehat{\text{COST}}^U[\text{IsRowAllOnes}] \\ &= W(N, \varepsilon_2) \cdot 2 \cdot W_U(M, \varepsilon_1) \cdot 2 \cdot \widehat{\text{COST}}^U[A] \end{aligned}$$

Havoc Cost: Upper-bound on program execution with errors.

Goal: Capture worst-case cost in the presence of errors.

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes( $i : \text{Fin}\langle N \rangle$ )  $\rightarrow \text{Bool}$  do  
   $b \leftarrow \text{all}[A(i, \_)]$ ;  
  return  $b$ 
```

```
fn HasAllOnesRow()  $\rightarrow \text{Bool}$  do  
   $b, i \leftarrow \text{search}[\text{IsRowAllOnes}(\_)]$ ;  
  return  $b$ 
```

$$\begin{aligned} & \widehat{\text{HAVOC}}^Q[\text{HasAllOnesRow}] \\ &= W(N, \varepsilon_2) \cdot 2 \cdot \widehat{\text{COST}}^U[\text{IsRowAllOnes}] \\ &= W(N, \varepsilon_2) \cdot 2 \cdot W_U(M, \varepsilon_1) \cdot 2 \cdot \widehat{\text{COST}}^U[A] \end{aligned}$$

Input-dependent Expected Cost Analysis

Motivation: Estimate the advantage given an input.

Grover's Search: Expected cost, $O(\sqrt{N/K})$, better than worst-case, $O(\sqrt{N})$.

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes( $i : \text{Fin}\langle N \rangle$ )  $\rightarrow \text{Bool}$  do  
   $b \leftarrow \text{all}[A(i, \_)]$ ;  
  return  $b$ 
```

```
fn HasAllOnesRow()  $\rightarrow \text{Bool}$  do  
   $b, i \leftarrow \text{search}[\text{IsRowAllOnes}(\_)]$ ;  
  return  $b$ 
```

$$\begin{aligned} & \widehat{\text{EXPCOST}}^{\mathcal{Q}}[\text{HasAllOnesRow}]() \\ &= E(N, K_A, \varepsilon_2) \cdot 2 \cdot \widehat{\text{COST}}^{\mathcal{U}}[\text{IsRowAllOnes}] \\ &= E(N, K_A, \varepsilon_2) \cdot 2 \cdot W_U(M, \varepsilon_1) \cdot 2 \cdot \widehat{\text{COST}}^{\mathcal{U}}[A] \end{aligned}$$

Input-dependent Expected Cost Analysis

Motivation: Estimate the advantage given an input.

Grover's Search: Expected cost, $O(\sqrt{N/K})$, better than worst-case, $O(\sqrt{N})$.

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes(i : Fin⟨N⟩) → Bool do
  b ← all[A(i, _)];
  return b
```

```
fn HasAllOnesRow() → Bool do
  b, i ← search[IsRowAllOnes(_)];
  return b
```

$$\begin{aligned}
 & \widehat{\text{EXPCOST}}^{\mathcal{Q}}[\text{HasAllOnesRow}]() \\
 &= E(N, K_A, \varepsilon_2) \cdot 2 \cdot \widehat{\text{COST}}^{\mathcal{U}}[\text{IsRowAllOnes}] \\
 &= E(N, K_A, \varepsilon_2) \cdot 2 \cdot W_U(M, \varepsilon_1) \cdot 2 \cdot \widehat{\text{COST}}^{\mathcal{U}}[A]
 \end{aligned}$$

Input-dependent Expected Cost Analysis

Motivation: Estimate the advantage given an input.

Grover's Search: Expected cost, $O(\sqrt{N/K})$, better than worst-case, $O(\sqrt{N})$.

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes(i : Fin⟨N⟩) → Bool do
  b ← all[A(i, _)];
  return b
```

```
fn HasAllOnesRow() → Bool do
  b, i ← search[IsRowAllOnes(_)];
  return b
```

$$\begin{aligned}
 & \widehat{\text{EXPCOST}}^{\mathcal{Q}}[\text{HasAllOnesRow}]() \\
 &= E(N, K_A, \varepsilon_2) \cdot 2 \cdot \widehat{\text{COST}}^{\mathcal{U}}[\text{IsRowAllOnes}] \\
 &= E(N, K_A, \varepsilon_2) \cdot 2 \cdot W_U(M, \varepsilon_1) \cdot 2 \cdot \widehat{\text{COST}}^{\mathcal{U}}[A]
 \end{aligned}$$

Input-dependent Expected Cost Analysis

Motivation: Estimate the advantage given an input.

Grover's Search: Expected cost, $O(\sqrt{N/K})$, better than worst-case, $O(\sqrt{N})$.

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes( $i : \text{Fin}\langle N \rangle$ )  $\rightarrow \text{Bool}$  do
   $b \leftarrow \text{all}[A(i, \_)]$ ;
  return  $b$ 
```

```
fn HasAllOnesRow()  $\rightarrow \text{Bool}$  do
   $b, i \leftarrow \text{search}[\text{IsRowAllOnes}(\_)]$ ;
  return  $b$ 
```

$$\begin{aligned} & \widehat{\text{EXPCOST}}^{\mathcal{Q}}[\text{HasAllOnesRow}]() \\ &= E(N, K_A, \varepsilon_2) \cdot 2 \cdot \widehat{\text{COST}}^{\mathcal{U}}[\text{IsRowAllOnes}] \\ &= E(N, K_A, \varepsilon_2) \cdot 2 \cdot W_U(M, \varepsilon_1) \cdot 2 \cdot \widehat{\text{COST}}^{\mathcal{U}}[A] \end{aligned}$$

Input-dependent Expected Cost Analysis

Motivation: Estimate the advantage given an input.

Grover's Search: Expected cost, $O(\sqrt{N/K})$, better than worst-case, $O(\sqrt{N})$.

$A : (\text{Fin}\langle N \rangle, \text{Fin}\langle M \rangle) \rightarrow \text{Bool}$

```
fn IsRowAllOnes(i : Fin⟨N⟩) → Bool do
  b ← all[A(i, _)];
  return b
```

```
fn HasAllOnesRow() → Bool do
  b, i ← search[IsRowAllOnes(_)];
  return b
```

$$\begin{aligned}
 & \widehat{\text{EXPCOST}}^{\mathcal{Q}}[\text{HasAllOnesRow}]() \\
 &= E(N, K_A, \varepsilon_2) \cdot 2 \cdot \widehat{\text{COST}}^{\mathcal{U}}[\text{IsRowAllOnes}] \\
 &= E(N, K_A, \varepsilon_2) \cdot 2 \cdot W_U(M, \varepsilon_1) \cdot 2 \cdot \widehat{\text{COST}}^{\mathcal{U}}[A]
 \end{aligned}$$

Traq Overview

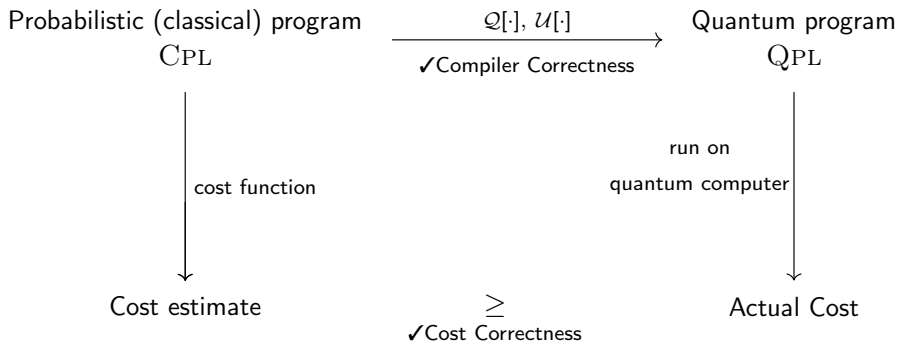


Figure: An outline of Traq.

Haskell Package: TRAQ

- TRAQ as a Haskell package, ≈ 9000 LOC.
- Polymorphic AST for adding primitives: `data Expr $\mathcal{P}$  = ... | PrimCall $\mathcal{P}$`
- Extensible: Add new primitives by implementing typeclasses.

```
class ExpCost  $\mathcal{P}$  where expCost ::  $\mathcal{P} \rightarrow \Sigma \rightarrow \text{Cost}$ 
```

Haskell Package: TRAQ

- TRAQ as a Haskell package, ≈ 9000 LOC.
- Polymorphic AST for adding primitives: `data Expr $\mathcal{P}$  = ... | PrimCall $\mathcal{P}$`
- Extensible: Add new primitives by implementing typeclasses.

```
class ExpCost  $\mathcal{P}$  where expCost ::  $\mathcal{P} \rightarrow \Sigma \rightarrow \text{Cost}$ 
```


Haskell Package: TRAQ

- TRAQ as a Haskell package, ≈ 9000 LOC.
- Polymorphic AST for adding primitives: `data Expr $\mathcal{P}$  = ... | PrimCall $\mathcal{P}$`
- Extensible: Add new primitives by implementing typeclasses.

```
class ExpCost  $\mathcal{P}$  where expCost ::  $\mathcal{P} \rightarrow \Sigma \rightarrow \text{Cost}$ 
```

Haskell Package: TRAQ

- TRAQ as a Haskell package, ≈ 9000 LOC.
- Polymorphic AST for adding primitives: `data Expr $\mathcal{P}$  = ... | PrimCall $\mathcal{P}$`
- Extensible: Add new primitives by implementing typeclasses.

```
class ExpCost  $\mathcal{P}$  where expCost ::  $\mathcal{P} \rightarrow \Sigma \rightarrow \text{Cost}$ 
```

Haskell Package: TRAQ

- TRAQ as a Haskell package, ≈ 9000 LOC.
- Polymorphic AST for adding primitives: `data Expr $\mathcal{P}$  = ... | PrimCall $\mathcal{P}$`
- Extensible: Add new primitives by implementing typeclasses.

```
class ExpCost  $\mathcal{P}$  where expCost ::  $\mathcal{P} \rightarrow \Sigma \rightarrow \text{Cost}$ 
```

Primitives in Traq

Primitive ($\mathcal{P}[\lambda]$)	$\text{EQQUERY}_{\mathcal{Q}\mathcal{P}_\varepsilon}^{\mathcal{U}}(\lambda)$	$\text{QUERY}_{\mathcal{U}\mathcal{P}_\varepsilon}^{\mathcal{U}}$
any/all/search	$\mathcal{O}(\sqrt{N/K})$ where $K = \{v \mid \lambda(v) = \mathbf{1}_1\} $ [Boyer et al.(1998), Cade et al.(2023)]	$\mathcal{O}(\sqrt{N} \log(1/\varepsilon))$ [Zalka(1999), Cade et al.(2023)]
amplify _{$p_{\min}$}	$2 \cdot \mathcal{O}(\frac{1}{\sqrt{p_{\text{good}}}})$ where $p_{\text{good}} = \Pr_{\lambda()}(1)$ [Boyer et al.(1998), Cade et al.(2023)]	$\mathcal{O}(\frac{1}{\sqrt{p_{\min}}} \ln(2/\varepsilon))$ [Yoder et al.(2014)]
simon _{p_{coll}}	$\mathcal{O}(n + \log(1/\varepsilon))$ [Kaplan et al.(2016)]	
min/max/argmin/argmax	$\mathcal{O}(\sqrt{N} \log(1/\varepsilon))$ [Durr and Hoyer(1999), Cade et al.(2023)]	

Primitives in Traq

Primitive ($\mathcal{P}[\lambda]$)	$\text{EQQUERY}_{\mathcal{Q}\mathcal{P}_\varepsilon}^{\mathcal{U}}(\lambda)$	$\text{QUERY}_{\mathcal{U}\mathcal{P}_\varepsilon}^{\mathcal{U}}$
any/all/search	$\mathcal{O}(\sqrt{N/K})$ where $K = \{v \mid \lambda(v) = \mathbf{1}_1\} $ [Boyer et al.(1998), Cade et al.(2023)]	$\mathcal{O}(\sqrt{N} \log(1/\varepsilon))$ [Zalka(1999), Cade et al.(2023)]
amplify _{$p_{\min}$}	$2 \cdot \mathcal{O}\left(\frac{1}{\sqrt{p_{\text{good}}}}\right)$ where $p_{\text{good}} = \Pr_{\lambda()}(1)$ [Boyer et al.(1998), Cade et al.(2023)]	$\mathcal{O}\left(\frac{1}{\sqrt{p_{\min}}} \ln(2/\varepsilon)\right)$ [Yoder et al.(2014)]
simon _{p_{coll}}	$\mathcal{O}(n + \log(1/\varepsilon))$ [Kaplan et al.(2016)]	
min/max/argmin/argmax	$\mathcal{O}(\sqrt{N} \log(1/\varepsilon))$ [Durr and Hoyer(1999), Cade et al.(2023)]	

Primitives in Traq

Primitive ($\mathcal{P}[\lambda]$)	$\text{EQQUERY}_{\mathcal{Q}\mathcal{P}_\varepsilon}^{\mathcal{U}}(\lambda)$	$\text{QUERY}_{\mathcal{U}\mathcal{P}_\varepsilon}^{\mathcal{U}}$
any/all/search	$\mathcal{O}(\sqrt{N/K})$ where $K = \{v \mid \lambda(v) = \mathbf{1}_1\} $ [Boyer et al.(1998), Cade et al.(2023)]	$\mathcal{O}(\sqrt{N} \log(1/\varepsilon))$ [Zalka(1999), Cade et al.(2023)]
amplify _{$p_{\min}$}	$2 \cdot \mathcal{O}(\frac{1}{\sqrt{p_{\text{good}}}})$ where $p_{\text{good}} = \Pr_{\lambda_0}(1)$ [Boyer et al.(1998), Cade et al.(2023)]	$\mathcal{O}(\frac{1}{\sqrt{p_{\min}}} \ln(2/\varepsilon))$ [Yoder et al.(2014)]
simon _{p_{coll}}	$\mathcal{O}(n + \log(1/\varepsilon))$ [Kaplan et al.(2016)]	
min/max/argmin/argmax	$\mathcal{O}(\sqrt{N} \log(1/\varepsilon))$ [Durr and Hoyer(1999), Cade et al.(2023)]	

Primitives in Traq

Primitive ($\mathcal{P}[\lambda]$)	$\text{EQQUERY}_{\mathcal{Q}\mathcal{P}_\varepsilon}^{\mathcal{U}}(\lambda)$	$\text{QUERY}_{\mathcal{U}\mathcal{P}_\varepsilon}^{\mathcal{U}}$
any/all/search	$\mathcal{O}(\sqrt{N/K})$ where $K = \{v \mid \lambda(v) = \mathbf{1}_1\} $ [Boyer et al.(1998), Cade et al.(2023)]	$\mathcal{O}(\sqrt{N} \log(1/\varepsilon))$ [Zalka(1999), Cade et al.(2023)]
amplify _{$p_{\min}$}	$2 \cdot \mathcal{O}(\frac{1}{\sqrt{p_{\text{good}}}})$ where $p_{\text{good}} = \Pr_{\lambda()}(1)$ [Boyer et al.(1998), Cade et al.(2023)]	$\mathcal{O}(\frac{1}{\sqrt{p_{\min}}} \ln(2/\varepsilon))$ [Yoder et al.(2014)]
simon _{p_{coll}}	$\mathcal{O}(n + \log(1/\varepsilon))$ [Kaplan et al.(2016)]	
min/max/argmin/argmax	$\mathcal{O}(\sqrt{N} \log(1/\varepsilon))$ [Durr and Hoyer(1999), Cade et al.(2023)]	

Primitives in Traq

Primitive ($\mathcal{P}[\lambda]$)	$\text{EQQUERY}_{\mathcal{Q}\mathcal{P}_\varepsilon}^{\mathcal{U}}(\lambda)$	$\text{QUERY}_{\mathcal{U}\mathcal{P}_\varepsilon}^{\mathcal{U}}$
any/all/search	$\mathcal{O}(\sqrt{N/K})$ where $K = \{v \mid \lambda(v) = \mathbf{1}_1\} $ [Boyer et al.(1998), Cade et al.(2023)]	$\mathcal{O}(\sqrt{N} \log(1/\varepsilon))$ [Zalka(1999), Cade et al.(2023)]
amplify _{$p_{\min}$}	$2 \cdot \mathcal{O}(\frac{1}{\sqrt{p_{\text{good}}}})$ where $p_{\text{good}} = \Pr_{\lambda()}(1)$ [Boyer et al.(1998), Cade et al.(2023)]	$\mathcal{O}(\frac{1}{\sqrt{p_{\min}}} \ln(2/\varepsilon))$ [Yoder et al.(2014)]
simon _{p_{coll}}	$\mathcal{O}(n + \log(1/\varepsilon))$ [Kaplan et al.(2016)]	
min/max/argmin/argmax	$\mathcal{O}(\sqrt{N} \log(1/\varepsilon))$ [Durr and Hoyer(1999), Cade et al.(2023)]	

Applications of TRAQ: Case-studies

Cryptanalysis

Program	Primitives	LoCs	Literature
3-ROUND FEISTEL ATTACK	simon	64	[Kaplan et al.(2016)]
EVEN-MANSOUR ATTACK	simon	25	[Kaplan et al.(2016)]

Optimization

Program	Primitives	LoCs	Literature
MAX-K-SAT	argmax/search	33	[Cade et al.(2023)]
0-1 KNAPSACK	amplify	98	[Wilkening et al.(2024)]

Search

Program	Primitives	LoCs	Literature
TRIANGLE FINDING	search	43	[Buhrman et al.(2005)]
FARTHEST POINTS	argmax	39	[Aïmeur et al.(2007)]
MATRIX SEARCH	search , all	18	[Høyer et al.(2003), Ambainis et al.(2010)]
DEPTH-3 NAND	all	28	[Høyer et al.(2003), Ambainis et al.(2010)]

Applications of TRAQ: Case-studies

Cryptanalysis

Program	Primitives	LoCs	Literature
3-ROUND FEISTEL ATTACK	simon	64	[Kaplan et al.(2016)]
EVEN-MANSOUR ATTACK	simon	25	[Kaplan et al.(2016)]

Optimization

Program	Primitives	LoCs	Literature
MAX-K-SAT	argmax/search	33	[Cade et al.(2023)]
0-1 KNAPSACK	amplify	98	[Wilkening et al.(2024)]

Search

Program	Primitives	LoCs	Literature
TRIANGLE FINDING	search	43	[Buhrman et al.(2005)]
FARTHEST POINTS	argmax	39	[Aïmeur et al.(2007)]
MATRIX SEARCH	search , all	18	[Høyer et al.(2003), Ambainis et al.(2010)]
DEPTH-3 NAND	all	28	[Høyer et al.(2003), Ambainis et al.(2010)]

Applications of TRAQ: Case-studies

Cryptanalysis

Program	Primitives	LoCs	Literature
3-ROUND FEISTEL ATTACK	simon	64	[Kaplan et al.(2016)]
EVEN-MANSOUR ATTACK	simon	25	[Kaplan et al.(2016)]

Optimization

Program	Primitives	LoCs	Literature
MAX-K-SAT	argmax/search	33	[Cade et al.(2023)]
0-1 KNAPSACK	amplify	98	[Wilkening et al.(2024)]

Search

Program	Primitives	LoCs	Literature
TRIANGLE FINDING	search	43	[Buhrman et al.(2005)]
FARTHEST POINTS	argmax	39	[Aïmeur et al.(2007)]
MATRIX SEARCH	search , all	18	[Høyer et al.(2003), Ambainis et al.(2010)]
DEPTH-3 NAND	all	28	[Høyer et al.(2003), Ambainis et al.(2010)]

Conclusion

Contributions:

- A classical (probabilistic) source language, CPL.
- Compilation from probabilistic source to quantum target.
- Cost analysis for source programs to estimate Quantum advantage.
- Formal guarantees on compilation and cost using error analysis.
- Haskell Package.

Future directions:

1. Support more cost models, e.g., Gate Complexity.
2. Enable support for more efficient quantum algorithms, e.g., variable-time quantum search.



arXiv



GitHub

Thank you!



Esma Aïmeur, Gilles Brassard, and Sébastien Gambs. 2007.

Quantum clustering algorithms. In *Proceedings of the 24th International Conference on Machine Learning* (Corvalis, Oregon, USA) (ICML '07). Association for Computing Machinery, New York, NY, USA, 1–8.

[doi:10.1145/1273496.1273497](https://doi.org/10.1145/1273496.1273497)



A. Ambainis, A. M. Childs, B. W. Reichardt, R. Špalek, and S. Zhang. 2010.

Any AND-OR Formula of Size N Can Be Evaluated in Time $N^{\{1/2+o(1)\}}$ on a Quantum Computer.

SIAM J. Comput. 39, 6 (2010), 2513–2530.

arXiv:<https://doi.org/10.1137/080712167> doi:10.1137/080712167



Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp. 1998.

Tight Bounds on Quantum Searching.

Fortschritte der Physik 46, 4-5 (1998), 493–505.

[doi:10.1002/\(SICI\)1521-3978\(199806\)46:4/5<493::AID-PROP493>3.0.CO;2-P](https://doi.org/10.1002/(SICI)1521-3978(199806)46:4/5<493::AID-PROP493>3.0.CO;2-P)



Harry Buhrman, Christoph Dürr, Mark Heiligman, Peter Høyer, Frédéric Magniez, Miklos Santha, and Ronald de Wolf. 2005.

Quantum Algorithms for Element Distinctness.

SIAM J. Comput. 34, 6 (Jan. 2005), 1324–1330.

doi:10.1137/s0097539702402780



Chris Cade, Marten Folkertsma, Ido Niesen, and Jordi Weggemans. 2023.

Quantifying Grover speed-ups beyond asymptotic analysis.

Quantum 7 (Oct. 2023), 1133.

doi:10.22331/q-2023-10-10-1133



Christoph Dürr and Peter Hoyer. 1999.

A Quantum Algorithm for Finding the Minimum.

arXiv:quant-ph/9607014 [quant-ph] <https://arxiv.org/abs/quant-ph/9607014>



Peter Høyer, Michele Mosca, and Ronald de Wolf. 2003.

Quantum Search on Bounded-Error Inputs.

Springer Berlin Heidelberg, 291–299.

doi:10.1007/3-540-45061-0_25



Marc Kaplan, Gaëtan Leurent, Anthony Leverrier, and María Naya-Plasencia. 2016.
Breaking Symmetric Cryptosystems Using Quantum Period Finding.

Springer Berlin Heidelberg, 207–237.

doi:10.1007/978-3-662-53008-5_8



Sören Wilkening, Andreea-Iulia Lefterovici, Lennart Binkowski, Michael Perk, Sándor Fekete,
and Tobias J. Osborne. 2024.

A quantum algorithm for solving 0-1 Knapsack problems.

arXiv:2310.06623 [quant-ph] <https://arxiv.org/abs/2310.06623>



Theodore J. Yoder, Guang Hao Low, and Isaac L. Chuang. 2014.

Fixed-Point Quantum Search with an Optimal Number of Queries.

Physical Review Letters 113, 21 (nov 2014).

doi:10.1103/physrevlett.113.210501



Christof Zalka. 1999.

A Grover-based quantum search of optimal order for an unknown number of marked elements.

[arXiv:quant-ph/9902049 \[quant-ph\]](https://arxiv.org/abs/quant-ph/9902049) <https://arxiv.org/abs/quant-ph/9902049>